

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО
ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В.
Г. ШУХОВА» (БГТУ им. В.Г. Шухова)**

Кафедра программного обеспечения вычислительной техники и
автоматизированных систем

Лабораторная работа №3

по дисциплине: Метрология, стандартизация и сертификация
программного обеспечения
тема: «Сложность программной системы»

Выполнил: ст. группы ПВ-223
Дмитриев Андрей
Александрович

Проверил:
Осипов Олег Васильевич

Вариант 2.

Цель работы: изучить теоретические сведения и получить практические навыки построения потокового графа подпрограмм и вычисления метрики цикломатической сложности.

Задание к работе

1. Реализовать программные реализации всех подпрограмм в соответствии с вариантом задания;
2. Для каждой из реализованных подпрограмм выполнить построение потокового графа;
3. Определить базовое множество независимых путей в каждом построенном потоковом графе;
4. Определить цикломатическую сложность для каждой подпрограммы;
5. Определить наборы тестов для каждой подпрограммы, инициирующие выполнение каждого пути из базового множества.
6. Выполнить пункты 1-5 для собственного программного проекта, который был рассмотрен в предыдущей лабораторной работе, например, курсового проекта по дисциплине «Базы данных». В качестве объекта исследования необходимо выбрать одну из подпрограмм собственного проекта.

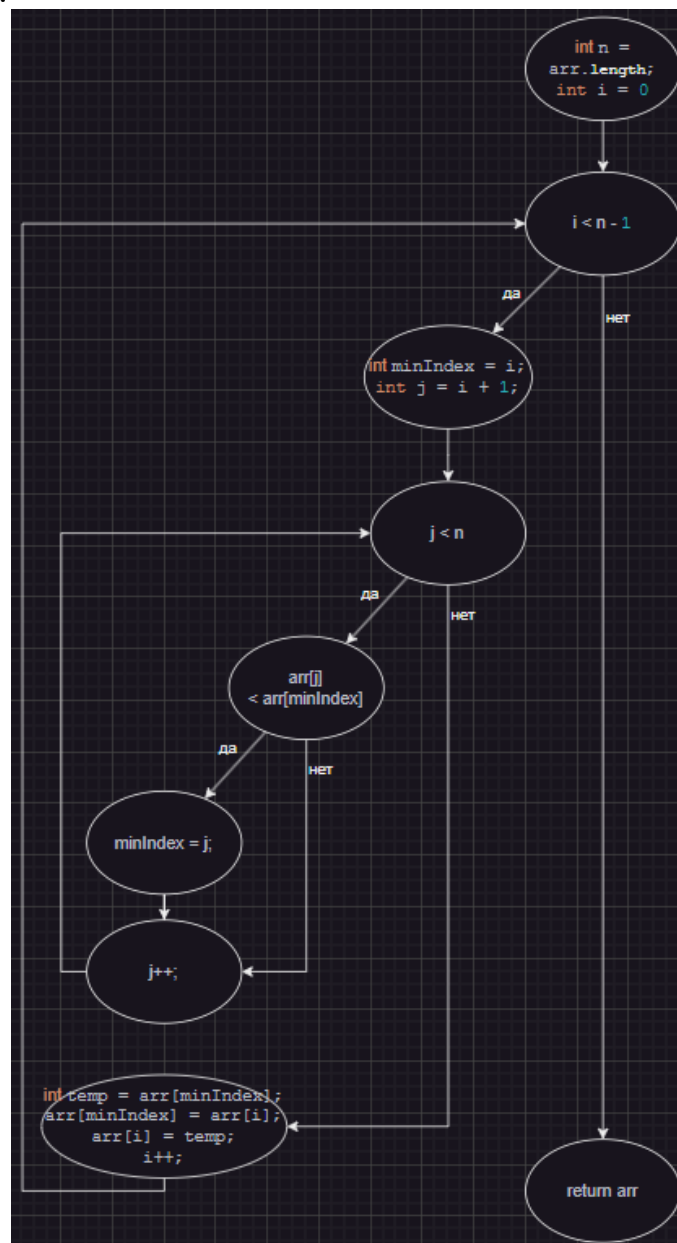
Ход работы:

Программа «Сортировка выбором»

Код программы:

```
void selectionSort(int[] arr) {  
    int n = arr.length;  
  
    for (int i = 0; i < n - 1; i++) {  
        int minIndex = i;  
        for (int j = i + 1; j < n; j++) {  
            if (arr[j] < arr[minIndex]) {  
                minIndex = j;  
            }  
        }  
  
        int temp = arr[minIndex];  
        arr[minIndex] = arr[i];  
        arr[i] = temp;  
    }  
}
```

Потоковый граф:



Базовое множество:

{[No], [Yes, Yes, No, No, No], [Yes, Yes, Yes, No, No], [Yes, No, No]}

Цикломатическая сложность: // не сходится

1) $V(G) = 4$

2) $V(G) = 11 - 9 + 2 = 4$

3) $V(G) = 3 + 1 = 4$

Набор тестов:

1) Пустой массив или из одного элемента

2) Массив из двух упорядоченных элементов

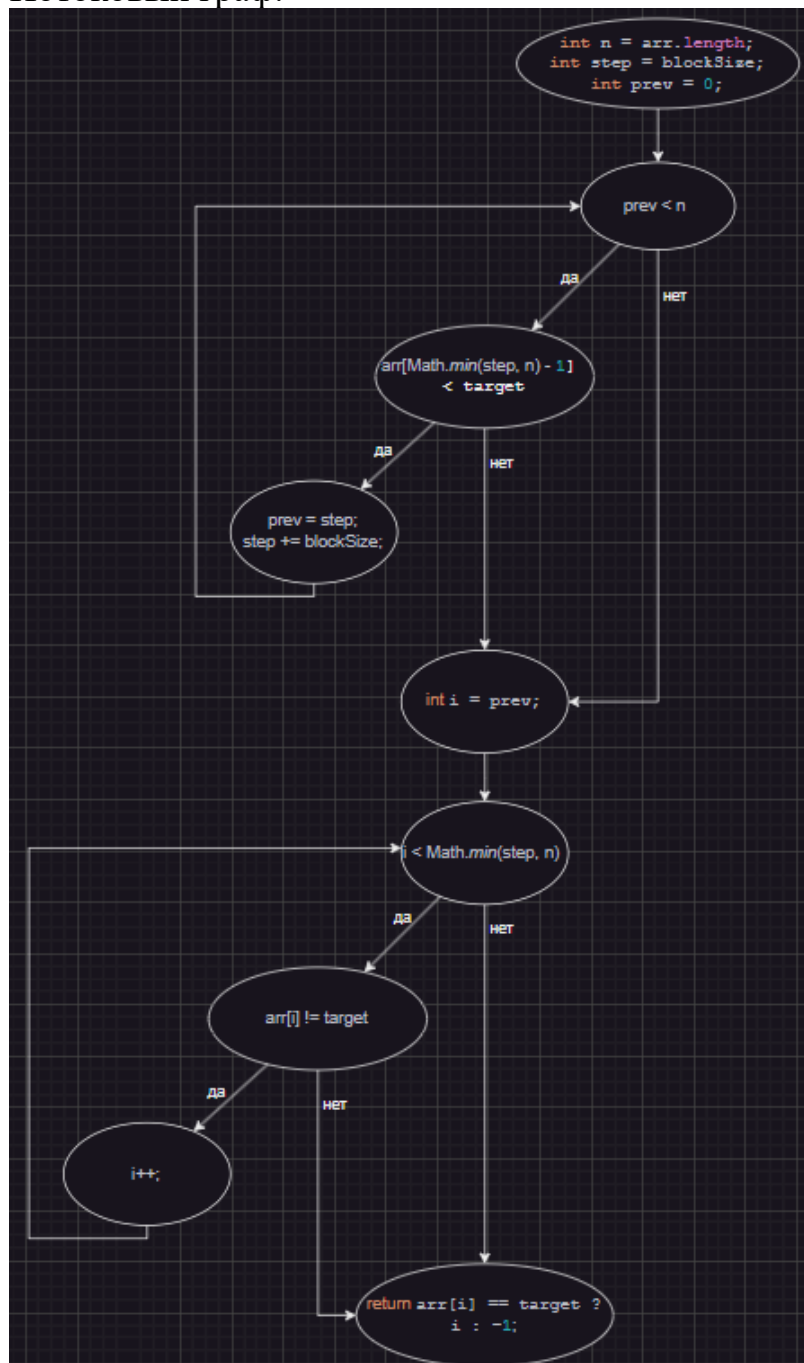
3) Массив из двух неупорядоченных элементов

4) Путь невозможно повторить

Программа «Блочный поиск»

```
int blockSearch(int[] arr, int blockSize, int target) {  
    int n = arr.length;  
    int step = blockSize;  
  
    int prev = 0;  
    while (prev < n && arr[Math.min(step, n) - 1] < target) {  
        prev = step;  
        step += blockSize;  
    }  
  
    int i = prev;  
    while (i < Math.min(step, n) && arr[i] != target) {  
        i++;  
    }  
  
    return arr[i] == target ? i : -1;  
}
```

Потоковый граф:



Базовое множество:

{[No,No], [Yes,No,Yes,No], [Yes,No,Yes,Yes,No], [Yes,Yes,No,Yes,No],
[Yes,Yes,No,Yes,Yes,No]}

Цикломатическая сложность:

- 1) $V(G) = 5$
- 2) $V(G) = 12 - 9 + 2 = 5$
- 3) $V(G) = 4 + 1 = 5$

Набор тестов:

- 1) Пустой массив
- 2) target=3, arr=[3, 4, ...]
- 3) blockSize=2+, target=3, arr=[1,3,4, ...]
- 4) blockSize=2+, target=3, arr=[1,1,3,4, ...]
- 5) blockSize=2+, target=3, arr=[1,1,1,3,4, ...]

Программа «Разбиение текста по строчкам»

```
private fun wrapText(
    text: String,
    paint: Paint,
    maxWidth: Float,
    maxHeight: Float
): List<String> {
    val words = text
        .replace("\n", " \n ")
        .split(" ")
    val lines = mutableListOf<String>()
    var currentLine = ""

    for (word in words) {
        if (word.isEmpty()) continue
        if (word == "\n") {
            lines.add(currentLine)
            currentLine = ""
            continue
        }

        val testLine = if (currentLine.isEmpty()) word else "$currentLine
$word"
        val testWidth = paint.measureText(testLine)

        currentLine = if (testWidth > maxWidth) {
            lines.add(currentLine)
            word
        } else {
            testLine
        }
    }

    if (currentLine.isNotEmpty()) {
        lines.add(currentLine)
    }

    val endByHeight = maxHeight / paint.textSize
    return if (endByHeight < lines.size) lines.subList(0,
endByHeight.toInt()) else lines
}
```

Упрощённая программа «Разбиение текста по строчкам»:

```
private fun wrapText(
    text: String,
    paint: Paint,
    maxWidth: Float,
    maxHeight: Float,
): List<String> {
    val words = text
        .replace("\n", " \n ")
        .split(" ")
    val lines = mutableListOf<String>()
    var currentLine = ""

    var i = 0
    while (i < words.size) {
        if (words[i].isNotEmpty()) {
            if (words[i] == "\n") {
                lines.add(currentLine)
                currentLine = ""
            } else {
                val testLine = if (currentLine.isEmpty()) words[i]
                    else "$currentLine ${words[i]}"
            }
        }
        i++
    }
    lines.add(currentLine)
    return lines
}
```

```

        val testWidth = paint.measureText(testLine)

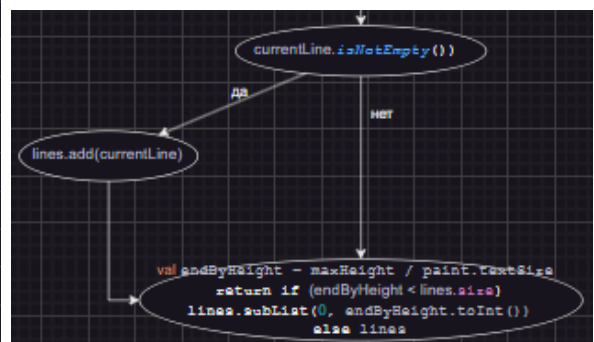
        if (testWidth > maxWidth) {
            lines.add(currentLine)
            currentLine = words[i]
        } else {
            currentLine = testLine
        }
    }
    i++
}

if (currentLine.isNotEmpty()) {
    lines.add(currentLine)
}

val endByHeight = maxHeight / paint.textSize
return if (endByHeight < lines.size) lines.subList(0,
endByHeight.toInt()) else lines
}

```

Потоковый граф:



Базовое множество:

{[Yes,No,No,No], [Yes,Yes,No,No,No,Yes], [Yes,Yes,No,Yes,No,Yes],
[Yes,Yes,Yes,No,No,No], [Yes,Yes,Yes,No,Yes,Yes,No,Yes,No,Yes],
[Yes,No,No,Yes]}

Цикломатическая сложность:

- 1) $V(G) = 6$
- 2) $V(G) = 17 - 13 + 2 = 6$
- 3) $V(G) = 5 + 1 = 6$

Набор тестов:

- 1) Пустая строка
- 2) Слово, которое помещается в одну строку
- 3) Два слова, второе из которых не влезает вместе с первым на одну строку
- 4) Перенос
- 5) Перенос и длинное слово
- 6) Перенос, длинное слово, перенос

Вывод: в ходе работы изучены функционально-ориентированные метрики программного обеспечения; получены практические навыки использования функционально ориентированных метрик ПО для оценки программного проекта.